

LAPORAN PRAKTIKUM 4
STRUKTUR DATA
IMPLEMENTASI ABSTRACT DATA TYPE (ADT) QUEUE
MENGGUNAKAN ARRAY



Disusun Oleh:

Silva Nurifvan

2511533004

Dosen Pengampu:

Wahyudi. Dr. S.T.M.T

Asisten Praktikum:

Sherly Sukmadira

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, laporan praktikum yang berjudul “Laporan Praktikum 4 Struktur Data: Implementasi Abstract Data Type (ADT) Queue Menggunakan Array” ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas praktikum pada mata kuliah Struktur Data. Adapun tujuan dari praktikum ini adalah untuk memahami konsep Abstract Data Type (ADT), khususnya pada struktur data Queue, serta menerapkannya menggunakan Array dan ArrayList dalam bahasa pemrograman Java. Selain itu, praktikum ini juga bertujuan agar mahasiswa dapat mengerti cara kerja antrian data dengan prinsip First In First Out (FIFO) dalam menyelesaikan suatu permasalahan.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Akhir kata, penulis berharap semoga laporan ini dapat bermanfaat serta menambah wawasan bagi pembaca mengenai struktur data, khususnya penerapan Queue dalam pemrograman.

Padang, 28 April 2026

Penulis

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan bagian yang sangat penting dalam ilmu komputer yang digunakan untuk mengatur dan menyimpan data agar dapat diproses secara efisien. Salah satu struktur data yang sering digunakan adalah Queue atau antrian. Queue adalah struktur data yang menerapkan prinsip First In First Out (FIFO), artinya data yang pertama kali masuk akan menjadi data pertama yang keluar. Konsep ini banyak diterapkan dalam kehidupan sehari-hari, seperti antrian di kasir, antrian kendaraan, maupun sistem antrian pada komputer.

Dalam pemrograman, Queue sangat berguna untuk mengatur proses yang berjalan secara berurutan. Penerapan Queue dapat dilakukan dengan berbagai cara, salah satunya adalah dengan menggunakan Array. Penggunaan Array dalam Queue dipilih karena susunan datanya teratur dan mudah dipahami, sehingga sangat cocok digunakan untuk mempelajari dasar konsep antrian data. Oleh karena itu, praktikum ini dilakukan agar mahasiswa dapat memahami cara kerja Queue menggunakan Array serta mampu mengimplementasikannya dalam bahasa pemrograman Java.

1.2 Tujuan

1. Memahami konsep dasar dari struktur data Queue.
2. Mengimplementasikan Queue dengan menggunakan Array dalam bahasa pemrograman Java.
3. Mengetahui prinsip kerja Queue dengan metode FIFO (First In First Out).
4. Melatih kemampuan pemrograman dalam menyelesaikan masalah menggunakan struktur data Queue.

1.3 Manfaat

1. Menambah pemahaman mengenai struktur data Queue.
2. Menjadi dasar pembelajaran untuk memahami struktur data lain yang lebih kompleks.
3. Membantu dalam memahami pengelolaan data secara berurutan di dalam program.
4. Meningkatkan kemampuan dalam mengimplementasikan Queue menggunakan Array.

BAB 2

PEMBAHASAN

Dalam pemrograman, struktur data merupakan cara untuk menyimpan dan mengatur data agar dapat digunakan secara efisien. Salah satu konsep penting dalam struktur data adalah Abstract Data Type (ADT). ADT adalah tipe data abstrak yang mendefinisikan sekumpulan nilai dan operasi yang bisa dilakukan terhadap data tersebut tanpa perlu menjelaskan secara rinci bagaimana cara mengimplementasikannya.

Konsep utama dari ADT adalah pemisahan antara interface (apa yang bisa dilakukan) dan implementasi (bagaimana cara melakukannya). Dengan menggunakan ADT, programmer bisa lebih fokus pada fungsi dan kegunaan suatu tipe data tanpa harus memahami detail cara kerjanya di dalam. Hal ini membuat program menjadi lebih modular, mudah dipelihara, dan mudah dikembangkan.

Di dalam bahasa pemrograman Java, ADT biasanya direpresentasikan menggunakan class. Sebuah class berisi atribut (data) dan metode (fungsi) yang mendefinisikan perilaku dari objek tersebut. Atribut digunakan untuk menyimpan data, sedangkan metode digunakan untuk melakukan operasi terhadap data tersebut.

Pada praktikum ini, ADT yang diterapkan adalah ADT Queue dengan menggunakan struktur data Array. Queue merupakan struktur data linear yang menerapkan prinsip FIFO (First In First Out), yaitu elemen yang pertama dimasukkan akan menjadi elemen pertama yang dikeluarkan. Struktur ini digunakan untuk mengelola data dengan mekanisme penambahan data dilakukan di bagian belakang (rear) dan penghapusan data dilakukan di bagian depan (front).

Selain itu, ADT Queue juga dilengkapi dengan beberapa operasi dasar, antara lain:

1. Penambahan data (enqueue), yaitu memasukkan elemen ke bagian paling belakang queue.
2. Penghapusan data (dequeue), yaitu mengambil atau menghapus elemen dari bagian paling depan queue.
3. Melihat elemen terdepan (front/peek), yaitu menampilkan elemen yang berada di depan tanpa menghapusnya.
4. Melihat elemen terakhir (rear), yaitu menampilkan elemen yang berada di paling belakang queue.
5. Menampilkan seluruh isi queue (display), yaitu menampilkan semua data yang tersimpan di dalam queue secara berurutan.
6. Mengetahui jumlah elemen (size), yaitu menghitung berapa banyak data yang sedang tersimpan di dalam queue.
7. Pengecekan kondisi (isEmpty dan isFull), yaitu untuk mengetahui apakah queue dalam keadaan kosong atau sudah penuh.

Dalam penerapannya, Queue menggunakan Array yang bersifat statis, sehingga jumlah elemen yang bisa ditampung sudah ditentukan sejak awal. Penggunaan Array memiliki kelebihan yaitu kecepatan akses data dan penggunaan memori yang efisien. Namun, kapasitas Array terbatas sehingga perlu pengelolaan indeks front dan rear dengan baik agar proses penambahan dan penghapusan data dapat berjalan dengan benar.

Dengan mempelajari cara mengimplementasikan ADT Queue menggunakan Array, mahasiswa dapat mengembangkan kemampuan dalam mengelola data secara terstruktur serta meningkatkan pemahaman terhadap konsep dasar struktur data dalam pemrograman Java.

1. Langkah Kerja

1.1 QueueArray_2511533004

Kode Program

```
1 package pekan4_2511533004;
2
3 public class QueueArray_2511533004 {
4     int front_3004, rear_3004, size_3004;
5     int capacity;
6     int array[];
7
8     public QueueArray_2511533004(int capacity) {
9         this.capacity = capacity;
10        front_3004 = this.size_3004 = 0;
11        rear_3004 = capacity - 1;
12        array = new int[this.capacity];
13    }
14
15    boolean isFull_3004(QueueArray_2511533004 queue) {
16        return (queue.size_3004 == queue.capacity);
17    }
18
19    boolean isEmpty_3004(QueueArray_2511533004 queue) {
20        return (queue.size_3004 == 0);
21    }
22
23    void enqueue_3004(int item) {
24        if (isFull_3004(this))
25            return;
26        this.rear_3004 = (this.rear_3004 + 1) % this.capacity;
27        this.array[this.rear_3004] = item;
28        this.size_3004 = this.size_3004 + 1;
29        System.out.println(item + " enqueued to queue");
30    }
31
32    int dequeue_3004() {
33        if (isEmpty_3004(this))
34            return Integer.MIN_VALUE;
35
36        int item = this.array[this.front_3004];
```

```

36     this.front_3004 = (this.front_3004 + 1) % this.capacity;
37     this.size_3004 = this.size_3004 - 1;
38     return item;
39 }
40
41 int front_3004() {
42     if (isEmpty_3004(this))
43         return Integer.MIN_VALUE;
44
45     return this.array[this.front_3004];
46 }
47
48 int rear_3004() {
49     if (isEmpty_3004(this))
50         return Integer.MIN_VALUE;
51
52     return this.array[this.rear_3004];
53 }
54
55 // mencetak elemen antrian
56 void display() {
57     int i;
58     if (front_3004 == rear_3004) {
59         System.out.printf("\nAntrian Kosong\n");
60         return;
61     }
62     // kunjungi dari belakang dan cetak
63     for (i = front_3004; i < rear_3004; i++) {
64         System.out.printf(" %d <-- ", array[i]);
65     }
66     return;
67 }
68 }

```

Pembahasan

Program ini dibuat dengan mendefinisikan class QueueArray_2511533004 yang digunakan untuk mengimplementasikan struktur data Queue menggunakan array. Queue merupakan struktur data linear yang menerapkan prinsip FIFO (First In First Out), yaitu elemen yang pertama masuk akan menjadi elemen pertama yang keluar.

1. Class ini memiliki atribut front, rear, dan size yang digunakan untuk mengatur posisi elemen dalam antrian. Variabel front menunjukkan posisi elemen terdepan, rear menunjukkan posisi elemen terakhir, dan size menunjukkan jumlah elemen yang tersimpan. Selain itu, atribut capacity digunakan untuk menentukan kapasitas maksimum queue, sedangkan array[] digunakan sebagai media penyimpanan data.

```

3 public class QueueArray_2511533004 {
4     int front_3004, rear_3004, size_3004;
5     int capacity;
6     int array[];

```

2. Constructor `QueueArray_2511533004 (int capacity)` digunakan untuk menginisialisasi queue dengan kapasitas tertentu. Pada constructor ini, nilai `front` dan `size` diatur menjadi 0, `rear` diatur ke `capacity - 1`, serta array dibuat sesuai kapasitas yang diberikan.

```
8 public QueueArray_2511533004(int capacity) {
9     this.capacity = capacity;
10    front_3004 = this.size_3004 = 0;
11    rear_3004 = capacity - 1;
12    array = new int[this.capacity];
13 }
```

3. Method `isFull_2511533004 (QueueArray_2511533004 queue)` digunakan untuk mengecek apakah queue dalam kondisi penuh, yaitu ketika jumlah elemen sama dengan kapasitas maksimum. Method `isEmpty_2511533004(QueueArray_2511533004 queue)` digunakan untuk mengecek apakah queue kosong, yaitu ketika jumlah elemen bernilai 0.

```
15 boolean isFull_3004(QueueArray_2511533004 queue) {
16     return (queue.size_3004 == queue.capacity);
17 }
18
19 boolean isEmpty_3004(QueueArray_2511533004 queue) {
20     return (queue.size_3004 == 0);
21 }
```

4. Method `enqueue_2511533004(int item)` digunakan untuk menambahkan elemen ke dalam queue. Sebelum penambahan, dilakukan pengecekan apakah queue penuh. Jika masih tersedia ruang, posisi `rear` diperbarui menggunakan operasi modulus agar membentuk circular queue, kemudian elemen dimasukkan ke array dan nilai `size` ditambah satu.

```
23 void enqueue_3004(int item) {
24     if (isFull_3004(this))
25         return;
26     this.rear_3004 = (this.rear_3004 + 1) % this.capacity;
27     this.array[this.rear_3004] = item;
28     this.size_3004 = this.size_3004 + 1;
29     System.out.println(item + " enqueued to queue");
30 }
```

5. Method `dequeue_2511533004()` digunakan untuk mengambil sekaligus menghapus elemen terdepan dari queue. Jika queue kosong, method mengembalikan `Integer.MIN_VALUE` sebagai penanda. Jika tidak kosong, elemen pada posisi `front` diambil, kemudian posisi `front` digeser ke indeks berikutnya menggunakan modulus dan nilai `size` dikurangi satu.

```
1 int dequeue_3004() {
2     if (isEmpty_3004(this))
3         return Integer.MIN_VALUE;
4
5     int item = this.array[this.front_3004];
6     this.front_3004 = (this.front_3004 + 1) % this.capacity;
7     this.size_3004 = this.size_3004 - 1;
8     return item;
9 }
```

6. Method `front_2511533004 ()` digunakan untuk melihat elemen paling depan tanpa menghapusnya, sedangkan method `rear_2511533004 ()` digunakan untuk melihat elemen paling belakang. Kedua method ini terlebih dahulu memeriksa apakah queue kosong.

```
41  int front_3004() {
42      if (isEmpty_3004(this))
43          return Integer.MIN_VALUE;
44
45      return this.array[this.front_3004];
46  }
47
48  int rear_3004() {
49      if (isEmpty_3004(this))
50          return Integer.MIN_VALUE;
51
52      return this.array[this.rear_3004];
53  }
54
```

7. Method `display_2511533004 ()` digunakan untuk menampilkan seluruh isi antrian. Jika queue kosong, program menampilkan pesan bahwa antrian kosong. Jika tidak kosong, elemen dicetak mulai dari posisi front menuju rear.

```
55  // mencetak elemen antrian
56  void display() {
57      int i;
58      if (front_3004 == rear_3004) {
59          System.out.printf("\nAntrian Kosong\n");
60          return;
61      }
62      // kuniungi dari belakang dan cetak
63      for (i = front_3004; i < rear_3004; i++) {
64          System.out.printf(" %d <-- ", array[i]);
65      }
66      return;
67  }
68 }
```

8. Untuk menjalankan program `QueueArray_2511533004` ini harus dilanjutkan dengan Class Driver `QueueArrayDriver_2511533004`.

1.2 QueueArrayDriver_2511533004

Kode Program

```
1 package pekan4_2511533004;
2
3 public class QueueArrayDriver_2511533004 {
4     public static void main(String[] args) {
5         QueueArray_2511533004 queue = new QueueArray_2511533004(1000);
6
7         queue.enqueue_3004(10);
8         queue.enqueue_3004(20);
9         queue.enqueue_3004(30);
10        queue.enqueue_3004(40);
11
12        System.out.println("Item di depan " + queue.front_3004());
13        System.out.println("Item paling belakang " + queue.rear_3004());
14        System.out.println("tampilan queue");
15        queue.display();
16
17        System.out.println();
18        System.out.println(queue.dequeue_3004() + " dihapus dari queue");
19
20        System.out.println("item di depan:" + queue.front_3004());
21        System.out.println("item dibelakang: " + queue.rear_3004());
22        System.out.println("tampilan queue setelah satu data dihapus");
23        queue.display();
24    }
25 }
```

Pembahasan

Program ini dibuat dengan mendefinisikan class QueueArrayDriver_2511533004 yang berfungsi sebagai driver untuk menguji implementasi struktur data Queue berbasis array pada class QueueArray_2511533004. Pada method main, dibuat objek queue dengan kapasitas 1000 elemen menggunakan constructor QueueArray_2511533004 (1000).

1. Method enqueue_3004 () digunakan untuk menambahkan data ke dalam queue, yaitu nilai 10, 20, 30, dan 40 secara berurutan. Proses ini menunjukkan prinsip FIFO (First In First Out), di mana data yang pertama masuk akan berada di posisi depan antrian.

```
3 public class QueueArrayDriver_2511533004 {
4     public static void main(String[] args) {
5         QueueArray_2511533004 queue = new QueueArray_2511533004(1000);
6
7         queue.enqueue_3004(10);
8         queue.enqueue_3004(20);
9         queue.enqueue_3004(30);
10        queue.enqueue_3004(40);
11    }
```

2. Method `front_3004 ()` digunakan untuk menampilkan elemen yang berada di posisi depan queue tanpa menghapusnya, sedangkan method `rear_3004 ()` digunakan untuk menampilkan elemen yang berada di posisi paling belakang.

```
System.out.println("Item di depan " + queue.front_3004());
System.out.println("Item paling belakang " + queue.rear_3004());
System.out.println("tampilan queue");
```

3. Method `display ()` digunakan untuk menampilkan seluruh isi queue dari elemen depan hingga elemen belakang, sehingga pengguna dapat melihat urutan data yang tersimpan di dalam antrian.

```
queue.display();

System.out.println();
```

4. Method `dequeue_3004 ()` digunakan untuk mengambil sekaligus menghapus elemen terdepan dari queue. Setelah satu data dihapus, program kembali menampilkan elemen depan, elemen belakang, serta isi queue terbaru untuk menunjukkan perubahan setelah proses penghapusan.

```
System.out.println("item di depan:" + queue.front_3004());
System.out.println("item dibelakang: " + queue.rear_3004());
System.out.println("tampilan queue setelah satu data dihapus");
queue.display();
}
```

5. Hasil Output dari program

```
<terminated> QueueArrayDriver_2511533004 [Java Applica
10 enqueued to queue
20 enqueued to queue
30 enqueued to queue
40 enqueued to queue
Item di depan 10
Item paling belakang 40
tampilan queue
 10 <--  20 <--  30 <--
10 dihapus dari queue
item di depan:20
item dibelakang: 40
tampilan queue setelah satu data dihapus
 20 <--  30 <--
```

1.3 QueueLinkedList_2511533004

Kode Program

```
1 package pekan4_2511533004;
2
3 import java.util.LinkedList;
4 import java.util.Queue;
5
6 public class QueueLinkedList_2511533004 {
7     public static void main(String[] args) {
8         Queue<Integer> q_3004 = new LinkedList<>();
9
10        // tambah elemen {0, 1, 2, 3, 4,5} ke antrian
11        for (int i = 0; i < 6; i++)
12            q_3004.add(i);
13
14        // Menampilkan isi antrian.
15        System.out.println("Elemen Antrian " + q_3004);
16
17        // Untuk menghapus kepala antrian.
18        int hapus = q_3004.remove();
19        System.out.println("Hapus elemen = " + hapus);
20        System.out.println(q_3004);
21
22        // Untuk melihat antrian terdapan
23        int depan = q_3004.peek();
24        System.out.println("Kepala Antrian = " + depan);
25
26        int banyak = q_3004.size();
27        System.out.println("Size Antrian = " + banyak);
28    }
29 }
```

Pembahasan

Program ini diawali dengan deklarasi package pekan4_2511533004 serta import java.util.LinkedList dan java.util.Queue yang digunakan untuk memanfaatkan struktur data Queue berbasis LinkedList. Program ini dibuat dengan class QueueLinkedList_2511533004 yang berfungsi untuk mendemonstrasikan penggunaan Queue bawaan Java.

1. Pada method main, dibuat objek Queue q_3004 = new LinkedList<>() yang digunakan untuk menyimpan data bertipe Integer. Interface Queue dipadukan dengan implementasi LinkedList sehingga antrian dapat dikelola secara dinamis tanpa batas ukuran tetap seperti array.

```
6 public class QueueLinkedList_2511533004 {
7     public static void main(String[] args) {
8         Queue<Integer> q_3004 = new LinkedList<>();
9     }
```

2. Struktur perulangan for digunakan untuk menambahkan elemen 0 sampai 5 ke dalam queue menggunakan method add(). Proses ini menempatkan data secara berurutan sesuai prinsip FIFO (First In First Out), yaitu elemen pertama yang masuk akan berada di posisi depan antrian.

```
// tambah elemen {0, 1, 2, 3, 4,5} ke antrian
for (int i = 0; i < 6; i++)
    q_3004.add(i);
```

3. Perintah System.out.println("Elemen Antrian " + q_3004) digunakan untuk menampilkan seluruh isi queue setelah proses penambahan data, sehingga urutan elemen dalam antrian dapat dilihat secara langsung.

```
// Menampilkan isi antrian.
System.out.println("Elemen Antrian " + q_3004);
```

4. .Method remove() digunakan untuk mengambil sekaligus menghapus elemen yang berada di kepala atau bagian depan queue. Nilai yang dihapus disimpan ke variabel hapus, kemudian ditampilkan ke layar bersama isi queue terbaru setelah penghapusan.

```
// Untuk menghapus kepala antrian.
int hapus = q_3004.remove();
System.out.println("Hapus elemen = " + hapus);
System.out.println(q_3004);
```

5. Method peek() digunakan untuk melihat elemen terdepan tanpa menghapusnya. Nilai tersebut disimpan ke variabel depan dan ditampilkan sebagai kepala antrian saat ini.

```
// Untuk melihat antrian terdepan
int depan = q_3004.peek();
System.out.println("Kepala Antrian = " + depan);
```

6. Method size() digunakan untuk menghitung jumlah elemen yang tersisa di dalam queue setelah proses penghapusan, kemudian hasilnya disimpan ke variabel banyak dan ditampilkan ke layar.

```
int banyak = q_3004.size();
System.out.println("Size Antrian = " + banyak);
}
```

7. Hasil Output Program

```
Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus elemen = 0
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size Antrian = 5
```

1.4 ReverseData_2511533004

Kode Program

```
1 package pekan4_2511533004;
2
3 import java.util.LinkedList;
4 import java.util.Queue;
5 import java.util.Stack;
6
7 public class ReverseData_2511533004 {
8     public static void main(String[] args) {
9         Queue<Integer> q_3004 = new LinkedList<Integer>();
10
11         q_3004.add(1);
12         q_3004.add(2);
13         q_3004.add(3); // [1, 2, 3]
14
15         System.out.println("sebelum reverse" + q_3004);
16
17         Stack<Integer> s = new Stack<Integer>();
18
19         while (!q_3004.isEmpty()) { // Q -> S
20             s.push(q_3004.remove());
21         }
22
23         while (!s.isEmpty()) { // S -> Q
24             q_3004.add(s.pop());
25         }
26
27         System.out.println("sesudah reverse= " + q_3004); // [3, 2, 1]
28     }
29 }
```

Pembahasan

Program ini diawali dengan deklarasi package pekan4_2511533004 serta import java.util.LinkedList, java.util.Queue, dan java.util.Stack yang digunakan untuk memanfaatkan struktur data Queue dan Stack. Program ini dibuat dengan class ReverseData_2511533004 yang berfungsi untuk membalik urutan data pada queue menggunakan bantuan stack.

1. Pada method main, dibuat objek Queue q_3004 = new LinkedList() yang digunakan untuk menyimpan data bertipe Integer. Data 1, 2, dan 3 ditambahkan ke dalam queue menggunakan method add(), kemudian isi queue ditampilkan sebelum proses pembalikan data.

```
public class ReverseData_2511533004 {
    public static void main(String[] args) {
        Queue<Integer> q_3004 = new LinkedList<Integer>();

        q_3004.add(1);
        q_3004.add(2);
        q_3004.add(3); // [1, 2, 3]

        System.out.println("sebelum reverse" + q_3004);
```

2. Dibuat objek Stack `s_3004 = new Stack()` yang digunakan sebagai media sementara dalam proses pembalikan urutan elemen. Stack dipilih karena menerapkan prinsip LIFO (Last In First Out), sehingga elemen terakhir yang masuk akan keluar terlebih dahulu.

```
Stack<Integer> s = new Stack<Integer>();
```

3. Perulangan `while (!q_3004.isEmpty())` digunakan untuk memindahkan seluruh elemen dari queue ke stack. Data diambil dari queue menggunakan method `remove()` lalu dimasukkan ke stack menggunakan `push()`. Proses ini menyebabkan urutan elemen menjadi terbalik di dalam stack.

```
while (!q_3004.isEmpty()) { // Q -> S
    s.push(q_3004.remove());
}
```

4. Perulangan kedua `while (!s.isEmpty())` digunakan untuk memindahkan kembali seluruh elemen dari stack ke queue. Data diambil dari stack menggunakan `pop()` lalu dimasukkan kembali ke queue menggunakan `add()`. Karena stack bekerja dengan prinsip LIFO, maka urutan elemen di queue menjadi terbalik dari urutan semula.

```
while (!s.isEmpty()) { // S -> Q
    q_3004.add(s.pop());
}
```

5. Setelah proses selesai, isi queue ditampilkan kembali menggunakan `System.out.println()` untuk menunjukkan hasil pembalikan data.

```
System.out.println("sesudah reverse= " + q_3004); // [3, 2, 1]
}
```

6. Hasil Output Program

```
<terminated> ReverseData_251153300
sebelum reverse[1, 2, 3]
sesudah reverse= [3, 2, 1]
```

1.5 IterasiQueue_2511533004

Kode Program

```
1 package pekan4_2511533004;
2
3 import java.util.Iterator;
4 import java.util.LinkedList;
5 import java.util.Queue;
6
7 public class IterasiQueue_2511533004 {
8     public static void main(String args[]) {
9         Queue<String> q_3004 = new LinkedList<>();
10
11         q_3004.add("Praktikum");
12         q_3004.add("Struktur");
13         q_3004.add("Data");
14         q_3004.add("Dan");
15         q_3004.add("Algoritma");
16
17         Iterator<String> iterator = q_3004.iterator();
18
19         while (iterator.hasNext()) {
20             System.out.print(iterator.next() + " ");
21         }
22     }
23 }
```

Pembahasan

Program ini diawali dengan deklarasi package pekan4_2511533004 serta import java.util.Iterator, java.util.LinkedList, dan java.util.Queue yang digunakan untuk memanfaatkan struktur data Queue serta melakukan proses iterasi data. Program ini dibuat dengan class IterasiQueue_2511533004 yang berfungsi untuk menampilkan elemen queue menggunakan Iterator.

1. Pada method main, dibuat objek Queue q_3004 = new LinkedList<>() yang digunakan untuk menyimpan data bertipe String. Queue diimplementasikan menggunakan LinkedList sehingga data dapat dikelola secara dinamis.

```
Queue<String> q_3004 = new LinkedList<>();
```

2. Method add() digunakan untuk menambahkan beberapa elemen ke dalam queue, yaitu "Praktikum", "Struktur", "Data", dan "Algoritma". Data dimasukkan secara berurutan sesuai prinsip FIFO (First In First Out).

```
q_3004.add("Praktikum");
q_3004.add("Struktur");
q_3004.add("Data");
q_3004.add("Dan");
q_3004.add("Algoritma");
```

3. Objek Iterator iterator = q_3004.iterator() digunakan untuk mengambil iterator dari queue. Iterator berfungsi untuk menelusuri elemen satu per satu tanpa menghapus data yang ada di dalam queue.

```
Iterator<String> iterator = q_3004.iterator();
```

4. Perulangan while (iterator.hasNext()) digunakan untuk memeriksa apakah masih terdapat elemen berikutnya. Selama masih ada data, method iterator.next() digunakan untuk mengambil elemen selanjutnya dan menampilkannya menggunakan System.out.print().

```
    while (iterator.hasNext()) {  
        System.out.print(iterator.next() + " ");  
    }  
}
```

5. Hasil Output Program

```
<terminated> IterasiQueue_2511533004 [Java Applic  
Praktikum Struktur Data Dan Algoritma
```


KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa implementasi struktur data Queue menggunakan array dan LinkedList telah berhasil dilakukan dengan baik serta mampu merepresentasikan konsep Abstract Data Type (ADT) Queue. Kedua metode implementasi tersebut mendukung operasi dasar Queue, seperti penambahan data (enqueue), penghapusan data (dequeue), melihat elemen terdepan (front/peek), serta menampilkan isi antrian, yang semuanya bekerja berdasarkan prinsip FIFO (First In First Out). Pada implementasi menggunakan array, struktur data memiliki ukuran tetap sehingga memerlukan pengelolaan indeks secara manual serta penanganan kondisi penuh dan kosong. Namun, penggunaan array cukup efisien dalam pengaksesan data karena elemen disimpan secara berurutan di memori. Sedangkan pada implementasi menggunakan LinkedList, struktur data bersifat dinamis sehingga tidak memiliki batasan ukuran tetap dan lebih fleksibel dalam proses penambahan maupun penghapusan elemen. Penggunaan LinkedList juga mempermudah implementasi Queue karena telah tersedia method bawaan Java yang mendukung pengelolaan antrian. Melalui berbagai program yang telah dibuat, seperti Queue berbasis array, Queue menggunakan LinkedList, pembalikan data Queue menggunakan bantuan Stack, serta iterasi elemen Queue menggunakan Iterator, dapat disimpulkan bahwa struktur data Queue sangat berguna dalam menyelesaikan berbagai permasalahan pemrograman secara sistematis. Secara keseluruhan, praktikum ini memberikan pemahaman yang lebih mendalam mengenai konsep Queue, perbedaan implementasi menggunakan array dan LinkedList, serta penerapannya dalam pemrograman Java, sehingga dapat meningkatkan kemampuan dalam merancang dan mengelola struktur data secara efisien, dinamis, dan terstruktur.

TUGAS PEKAN 4

Topik : Implementasi Queue (FIFO)

Tema: antrian pelanggan di loket pelayanan

Nama Class “AntrianLoket_2511533004”

Kode Program

```
1 package pekan4_2511533004;
2 import java.util.Scanner;
3 import java.util.Stack;
4
5 public class AntrianLoket_2511533004 {
6
7     int max = 10;
8     String data[] = new String[max];
9     int rear = -1;
10    int jumlah = 0;
11
12    // Tambah Antrian
13    void enqueue_3004(String isi) {
14        if (isFull_3004()) {
15            System.out.println("Antrian penuh!");
16        } else {
17            rear++;
18            data[rear] = isi;
19            jumlah++;
20            System.out.println("Data berhasil ditambah");
21        }
22    }
23
24    // Hapus Antrian
25    void dequeue_3004() {
26        if (isEmpty_3004()) {
27            System.out.println("Antrian kosong!");
28        } else {
29            System.out.println("Data keluar: " + data[0]);
30
31            for (int i = 0; i < rear; i++) {
32                data[i] = data[i + 1];
33            }
34
35            data[rear] = null;
36            rear--;
37            jumlah--;
38        }
39    }
40
41    // Tampilkan Antrian
42    void display_3004() {
43        if (isEmpty_3004()) {
44            System.out.println("Antrian kosong!");
45        } else {
46            System.out.println("Isi antrian:");
47            for (int i = 0; i <= rear; i++) {
48                System.out.println((i + 1) + ". " + data[i]);
49            }
50        }
51    }
52
53    boolean isEmpty_3004() {
54        return jumlah == 0;
55    }
56
57    boolean isFull_3004() {
58        return jumlah == max;
59    }
60
61    // Reverse
62    void reverse_3004() {
63        if (isEmpty_3004()) {
64            System.out.println("Antrian kosong!");
65        } else {
66            Stack<String> stack_3004 = new Stack<>();
67
68            for (int i = 0; i <= rear; i++) {
69                stack_3004.push(data[i]);
70            }
71        }
72    }
73 }
```

```

71
72     for (int i = 0; i <= rear; i++) {
73         data[i] = stack_3004.pop();
74     }
75
76     System.out.println("Antrian berhasil dibalik");
77 }
78 }
79
80 public static void main(String[] args) {
81
82     Scanner input = new Scanner(System.in);
83     AntrianLoket_2511533004 q_3004 = new AntrianLoket_2511533004();
84
85     int pilih = 0;
86     String isi;
87
88     do {
89         System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
90         System.out.println("1. Tambah Antrian");
91         System.out.println("2. Hapus Antrian");
92         System.out.println("3. Tampilkan Antrian");
93         System.out.println("4. Reverse Antrian");
94         System.out.println("5. Keluar");
95         System.out.print("Pilih menu : ");
96
97         // VALIDASI INPUT
98         if (input.hasNextInt()) {
99             pilih = input.nextInt();
100             input.nextLine();
101         } else {
102             System.out.println("Input harus angka!");
103             input.nextLine(); // buang input salah
104             continue;
105         }
106
107         switch (pilih) {
108             case 1:
109                 System.out.print("Input data : ");
110                 isi = input.nextLine();
111                 q_3004.enqueue_3004(isi);
112                 break;
113
114             case 2:
115                 q_3004.dequeue_3004();
116                 break;
117
118             case 3:
119                 q_3004.display_3004();
120                 break;
121
122             case 4:
123                 q_3004.reverse_3004();
124                 break;
125
126             case 5:
127                 System.out.println("Program selesai");
128                 break;
129
130             default:
131                 System.out.println("Menu tidak tersedia");
132         }
133     } while (pilih != 5);
134 }
135 }
136 }

```

Pembahasan

Program ini dibuat dengan mendefinisikan class `AntrianLoket_2511533004` yang digunakan untuk mengimplementasikan struktur data Queue menggunakan array dalam kasus antrian loket. Pada bagian awal program dilakukan import `java.util.Scanner` untuk menerima input pengguna dan `java.util.Stack` yang digunakan pada proses pembalikan antrian.

1. Class ini memiliki atribut `max`, `data[]`, `front`, `rear`, dan `jumlah` yang digunakan untuk mengatur proses antrian. Variabel `max`

menunjukkan kapasitas maksimum antrian, data[] digunakan sebagai tempat penyimpanan data, front menunjukkan posisi elemen terdepan, rear menunjukkan posisi elemen terakhir, sedangkan jumlah menyimpan banyaknya data yang ada di dalam antrian.

```
5 public class AntrianLoket_2511533004 {  
6  
7     int max = 10;  
8     String data[] = new String[max];  
9     int rear = -1;  
10    int jumlah = 0;
```

2. Method enqueue_3004(String isi) digunakan untuk menambahkan data ke dalam antrian. Sebelum penambahan, dilakukan pengecekan menggunakan isFull_3004 () untuk memastikan antrian tidak penuh. Jika masih tersedia ruang, posisi rear bertambah satu, data dimasukkan ke array, dan jumlah data bertambah.

```
// Tambah Antrian  
void enqueue_3004(String isi) {  
    if (isFull_3004()) {  
        System.out.println("Antrian penuh!");  
    } else {  
        rear++;  
        data[rear] = isi;  
        jumlah++;  
        System.out.println("Data berhasil ditambah");  
    }  
}
```

3. Method dequeue_3004 () digunakan untuk mengambil sekaligus menghapus data terdepan dari antrian. Jika antrian kosong, program menampilkan pesan kesalahan. Jika tidak kosong, data pada posisi depan ditampilkan, kemudian seluruh elemen digeser ke kiri agar posisi data tetap berurutan, lalu nilai rear dan jumlah dikurangi.

```
// Hapus Antrian  
void dequeue_3004() {  
    if (isEmpty_3004()) {  
        System.out.println("Antrian kosong!");  
    } else {  
        System.out.println("Data keluar: " + data[0]);  
  
        for (int i = 0; i < rear; i++) {  
            data[i] = data[i + 1];  
        }  
  
        data[rear] = null;  
        rear--;  
        jumlah--;  
    }  
}
```

4. Method `display_3004 ()` digunakan untuk menampilkan seluruh isi antrian. Jika antrian kosong, program menampilkan pesan bahwa antrian kosong. Jika tidak kosong, seluruh data dicetak secara berurutan dari depan hingga belakang menggunakan perulangan.

```
// Tampilkan Antrian
void display_3004() {
    if (isEmpty_3004()) {
        System.out.println("Antrian kosong!");
    } else {
        System.out.println("Isi antrian:");
        for (int i = 0; i <= rear; i++) {
            System.out.println((i + 1) + ". " + data[i]);
        }
    }
}
```

5. Method `isEmpty_3004 ()` digunakan untuk mengecek apakah antrian kosong dengan memeriksa nilai jumlah, sedangkan method `isFull_3004 ()` digunakan untuk mengecek apakah antrian telah penuh sesuai kapasitas maksimum.

```
boolean isEmpty_3004() {
    return jumlah == 0;
}

boolean isFull_3004() {
    return jumlah == max;
}
```

6. Method `reverse_3004 ()` digunakan untuk membalik urutan data dalam antrian dengan bantuan struktur data Stack. Seluruh elemen antrian dimasukkan ke dalam stack menggunakan `push()`, kemudian diambil kembali menggunakan `pop()` dan disimpan ke array sehingga urutan data menjadi terbalik.

```
// Reverse
void reverse_3004() {
    if (isEmpty_3004()) {
        System.out.println("Antrian kosong!");
    } else {
        Stack<String> stack_3004 = new Stack<>();

        for (int i = 0; i <= rear; i++) {
            stack_3004.push(data[i]);
        }

        for (int i = 0; i <= rear; i++) {
            data[i] = stack_3004.pop();
        }

        System.out.println("Antrian berhasil dibalik");
    }
}
```

7. Pada method main, dibuat objek Scanner dan objek AntrianLoket_2511533004 sebagai antrian utama. Program menggunakan perulangan do-while untuk menampilkan menu secara berulang, dengan pilihan tambah antrian, hapus antrian, tampilkan antrian, reverse antrian, dan keluar dari program.

```
public static void main(String[] args) {  
  
    Scanner input = new Scanner(System.in);  
    AntrianLoket_2511533004 q_3004 = new AntrianLoket_2511533004();  
  
    int pilih = 0;  
    String isi;  
  
    do {  
        System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");  
        System.out.println("1. Tambah Antrian");  
        System.out.println("2. Hapus Antrian");  
        System.out.println("3. Tampilkan Antrian");  
        System.out.println("4. Reverse Antrian");  
        System.out.println("5. Keluar");  
        System.out.print("Pilih menu : ");  
  
        // VALIDASI INPUT  
        if (input.hasNextInt()) {  
            pilih = input.nextInt();  
            input.nextLine();  
        }  
    } while (pilih != 5);  
}
```

8. Struktur switch-case digunakan untuk menjalankan perintah sesuai pilihan pengguna. Setiap menu akan memanggil method yang sesuai sehingga proses pengelolaan antrian dapat dilakukan secara interaktif.

```
switch (pilih) {  
    case 1:  
        System.out.print("Input data : ");  
        isi = input.nextLine();  
        q_3004.enqueue_3004(isi);  
        break;  
  
    case 2:  
        q_3004.dequeue_3004();  
        break;  
  
    case 3:  
        q_3004.display_3004();  
        break;  
  
    case 4:  
        q_3004.reverse_3004();  
        break;  
  
    case 5:  
        System.out.println("Program selesai");  
        break;  
  
    default:  
        System.out.println("Menu tidak tersedia");  
        break;  
}  
} while (pilih != 5);  
}
```

9. Hasil Output dari Program
Menambahkan Antrian dengan pilih menu 1

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse Antrian
5. Keluar
Pilih menu : 1
Input data : silva
Data berhasil ditambah

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse Antrian
5. Keluar
Pilih menu : 1
Input data : jefan
Data berhasil ditambah

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse Antrian
5. Keluar
```

```
Pilih menu : 3
Isi antrian:
1. silva
2. jefan
3. pia

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse Antrian
5. Keluar
Pilih menu : 4
Antrian berhasil dibalik

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse Antrian
5. Keluar
Pilih menu : 2
Data keluar: pia

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
```

```
3. Tampilkan Antrian
4. Reverse Antrian
5. Keluar
Pilih menu : 5
Program selesai
```